Ethics of Deep Learning

Last modified: 2026-11-04

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission. His course has no module on ethics, so this page is new, exercises included.

1 Welcome

1.1 Plan for this lecture

Earlier chapters asked how well a network does. This lecture asks four other questions about a trained network:

1. whose errors it makes;
2. what it gives away about its training set[1];
3. what training it costs;
4. what its users should be told about it.

Ethics and Fairness[2] in the DATA 471 notes covers bias, definitions of fairness and the result that they cannot all hold at once; this lecture links to it rather than repeating it.

1.2 The digits

```
import numpy as np
import pandas as pd
import torch
from torch import nn
import matplotlib.pyplot as plt

digits = pd.read_csv("data/optdigits-test.csv", header=None)
x = torch.tensor(digits.iloc[:, :64].to_numpy(), dtype=torch.float32) / 16
y = torch.tensor(digits[64].to_numpy())
order = torch.randperm(len(x), generator=torch.Generator().manual_seed(481))
train_idx, val_idx = order[:1000], order[1000:]
n_train, n_val = len(train_idx), len(val_idx)
loss_fn = nn.CrossEntropyLoss()
default_width, default_steps = 64, 300


def fit(rows, width=default_width, seed=0, n_steps=default_steps):
```

[1] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[2] https://morrison-lab.github.io/lds/chapters/ethics.html

```
    # A network with one hidden layer, trained by full-batch Adam on the given rows.
    torch.manual_seed(seed)
    net = nn.Sequential(nn.Linear(64, width), nn.ReLU(), nn.Linear(width, 10))
    optimizer = torch.optim.Adam(net.parameters(), lr=0.01)
    for step in range(n_steps):
        optimizer.zero_grad()
        loss_fn(net(x[rows]), y[rows]).backward()
        optimizer.step()
    return net


def correct(net, rows):
    with torch.no_grad():
        return (net(x[rows]).argmax(dim=1) == y[rows]).float()


def accuracy(net, rows):
    return correct(net, rows).mean().item()


def digit_accuracy(net, rows):
    # The share of each digit's images that the network classifies correctly.
    return pd.Series(correct(net, rows).numpy()).groupby(y[rows].numpy()).mean()


print(n_train, n_val)
```

```
1000 797
```

The examples use the 1,797 digits of `data/optdigits-test.csv`[3], each an 8 by 8 image, written by 13 people [1]. 1,000 random images form a training set[4], and the other 797 form a validation set[5]. `fit()` trains a network with one hidden layer of ReLU units, 64 of them unless `width` says otherwise, on the given rows by 300 steps of full-batch Adam[6]. `accuracy()` gives the share of the rows that a network classifies correctly, and `digit_accuracy()` gives that share for each digit separately.

2 Who the errors fall on

2.1 Too few eights

Exercise 2.1 (Too few eights)**.** A network is trained on the training set[a] with all but 10 of its eights removed. Would its accuracy[b] on the whole validation set[c] show that it has trouble with eights?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set

Solution

Solution 2.1.

[3] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv
[4] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[5] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set
[6] https://morrison-lab.github.io/mds/optimization-gradient-descent.html#def-adam

```
train_y = y[train_idx]
eights = train_idx[train_y == 8]
few_rows = torch.cat([train_idx[train_y != 8], eights[:10]])
full_net = fit(train_idx)
few_net = fit(few_rows)
full_acc, few_acc = accuracy(full_net, val_idx), accuracy(few_net, val_idx)
full_eights = digit_accuracy(full_net, val_idx)[8]
few_eights = digit_accuracy(few_net, val_idx)[8]
n_eights_train = len(eights)
n_eights_val = (y[val_idx] == 8).sum().item()
print(full_acc, few_acc, full_eights, few_eights)
```

```
0.9598494172096252 0.9410288333892822 0.9054054 0.6891892
```

```
# Stop the render if the prose below stops being true.
assert full_acc - few_acc < 0.03
assert full_eights - few_eights > 0.15
```

Hardly. The training set[a] holds 100 eights. Trained on all of them, the network classifies 96.0% of the validation digits correctly; trained on 10 of them, 94.1%. On the 74 eights of the validation set[b] alone, its accuracy[c] falls from 91% to 69%. Eights are only 9% of the validation set, so a large drop on them is a small drop overall.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set
[c] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

Definition 2.1 (Disaggregated evaluation)**.** A **disaggregated evaluation** reports a model's performance separately for each group of its inputs, such as each group of people or each condition the inputs were recorded under, as well as over the whole evaluation set [2, Sec. 4.7].

Example 2.1 (Accuracy for each digit)**.**

```
full_by_digit = digit_accuracy(full_net, val_idx)
few_by_digit = digit_accuracy(few_net, val_idx)
by_digit = pd.DataFrame({
    "digit": range(10),
    "images": (y[val_idx][:, None] == torch.arange(10)).sum(dim=0).numpy(),
    "all eights": full_by_digit.to_numpy(),
    "10 eights": few_by_digit.to_numpy(),
})
worst_digit, best_digit = full_by_digit.idxmin(), full_by_digit.idxmax()
fewest_images, most_images = by_digit["images"].min(), by_digit["images"].max()
print(worst_digit, best_digit, fewest_images, most_images)
```

```
3 7 73 94
```

```
knitr::kable(py$by_digit, digits = 2)
```

Table 1: The accuracy[a] of the two networks of Solution 2.1 on each digit of the validation set[b].

digit	images	all eights	10 eights
0	73	0.99	1.00
1	94	0.98	0.99
2	83	0.98	0.99
3	76	0.89	0.91
4	78	0.95	0.96

5	81	0.96	0.94
6	76	0.99	0.99
7	81	1.00	0.99
8	74	0.91	0.69
9	81	0.95	0.94

Table 1 is a disaggregated evaluation of the two networks, with the digits as the groups. Removing eights lowers the accuracy[c] on eights most. Even the network trained on every eight does better on some digits than on others, from 89% on 3s to 100% on 7s. Each digit has between 73 and 94 images in the validation set[d], so one image moves a digit's accuracy by more than one percentage point, and differences of a point or two between digits could change with another random split.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set
[c] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set

Remark (The same failure with people)

Remark 2.1 (The same failure with people). A training set[a] with few eights is a small case of what Ethics and Fairness[b] in the DATA 471 notes calls representation bias: a group that appears less often in the training set[c] can get less of the network's accuracy[d]. Among its documented cases[e], Buolamwini and Gebru [3] evaluated commercial face-analysis systems separately on groups of faces and found error rate[f]s far higher for darker-skinned women than for lighter-skinned men; the benchmarks that the systems' accuracy was usually reported on held mostly lighter-skinned faces.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/ethics.html#sec-bias-sources
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[d] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[e] https://morrison-lab.github.io/lds/chapters/ethics.html#sec-documented-cases
[f] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate

3 What a network remembers

3.1 Guessing the training set

Exercise 3.1 (Guessing the training set)**.** Someone has a trained network and one digit image, but not the network's training set[a]. How could they guess whether that image was in the training set?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set

Solution

Solution 3.1. Compute the network's loss[a] on the image. Training pushes the loss down on the training images themselves, so an image with an unusually small loss is more likely to be one of them.

```
members, outsiders = order[:100], order[100:200]
target_width = 256
target = fit(members, width=target_width)
per_image_loss = nn.CrossEntropyLoss(reduction="none")


def losses(net, rows):
    with torch.no_grad():
        return per_image_loss(net(x[rows]), y[rows])


member_loss, outsider_loss = losses(target, members), losses(target, outsiders)
member_acc, outsider_acc = accuracy(target, members), accuracy(target, outsiders)
n_members = len(members)
loss_ratio = (outsider_loss.median() / member_loss.median()).item()
outsiders_above = (outsider_loss > member_loss.max()).float().mean().item()
member_log = member_loss.clamp(min=1e-8).log10()
outsider_log = outsider_loss.clamp(min=1e-8).log10()
n_members_floor = (member_log == -8).sum().item()
print(member_loss.median().item(), outsider_loss.median().item(), member_acc, outsider_acc)
```

```
2.47952248173533e-05 0.00013982271775603294 1.0 0.9200000166893005
```

```
# Stop the render if the figure's alt text stops being true.
assert -3.5 < member_log.max().item() < -2.5
assert n_members_floor > 0 and (member_log[member_log > -8] > -7.5).all()
assert 0.3 < outsiders_above < 0.5
assert outsider_log.max().item() < 2
```

```
bins = np.linspace(-8, 2, 41)
fig, ax = plt.subplots(figsize=(6, 3))
ax.hist(member_log.numpy(), bins=bins, alpha=0.6, label="trained on");
ax.hist(outsider_log.numpy(), bins=bins, alpha=0.6, label="not trained on");
ax.set_xlabel("log10 of the cross-entropy loss");
ax.set_ylabel("images");
ax.legend();
fig.tight_layout();
plt.show()
```

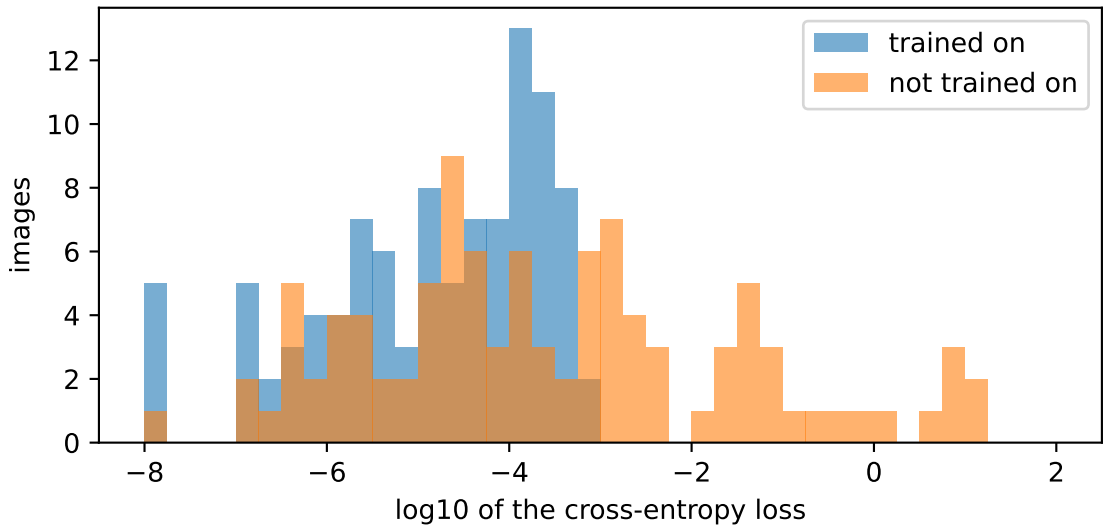


Figure 1: The loss[b] of one network on the 100 digits it was trained on and on 100 digits it was not. Losses below 10^{-8} are drawn at -8.

Here a network with a hidden layer of 256 units is trained on 100 digits, and compared on those and on 100 others. Its median cross-entropy loss[c] is 5.6 times larger on the digits it was not trained on. 41% of those other digits have a larger loss[d] than any digit it was trained on, as Figure 1 shows.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function
[c] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-cross-entropy-loss
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function

Definition 3.1 (Membership inference attack)**.** A **membership inference attack** uses a trained model's outputs on a data point to guess whether that point was in the model's training set[a] [4].

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set

Example 3.1 (Attacks on machine learning services)**.** Shokri et al. [4, Sec. I] ran membership inference attacks on classifiers trained by Google's and Amazon's machine learning[a] services, which do not reveal the models they train. Against those models, their attacks guessed membership with a median accuracy[b] of 94% and 74%. On a data set of hospital stays in Texas, where knowing that a record was in the training set[c] is itself sensitive, their attack was right over 70% of the time.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set

Exercise 3.2 (Where to draw the line)**.** The attacker of Exercise 3.1 knows how the network was trained: its architecture, its width[a], its number of training steps, and that it saw 100 digits. The attacker does not know which loss[b] counts as small for this network. How could the attacker choose a threshold?

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-width
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function

Solution

Solution 3.2. Train a shadow network of the same kind on 100 digits of the attacker's own, and take the largest loss[a] of the shadow network on those digits as the threshold. Then guess that an image was in the target's training set[b] when the target's loss on it is at most the threshold.

```
shadow_rows = order[1000:1100]
shadow = fit(shadow_rows, width=target_width, seed=1)
threshold = losses(shadow, shadow_rows).max()
flagged_members = (member_loss <= threshold).sum().item()
flagged_outsiders = (outsider_loss <= threshold).sum().item()
attack_acc = (flagged_members + n_members - flagged_outsiders) / (2 * n_members)
flagged_share_members = flagged_members / (flagged_members + flagged_outsiders)
print(threshold.item(), flagged_members, flagged_outsiders, attack_acc, flagged_share_members)
```

```
0.0010065733222290874 100 63 0.685 0.6134969325153374
```

```
# Stop the render if the prose below stops being true.
assert 0.6 < attack_acc < 0.8
```

This membership inference attack flags 100 of the target's 100 training digits and 63 of the 100 others. On the 200 digits together it guesses right 68% of the time, where a coin would guess right 50% of the time, but only 61% of the digits it flags were in the training set. It works because the target overfits[c]: it was trained on only 100 digits, is right on 100% of them and on 92% of the others, and the attacker knew how it was trained and had digits from the same 13 writers. Shokri et al. [4] train many shadow models and a classifier on their outputs, and Yeom et al. [5] guess from the loss alone; this solution joins the two ideas in the simplest way.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-overfitting

Remark (Overfitting and memorization)

Remark 3.1 (Overfitting and memorization). Yeom et al. [5, Sec. 3.2] show that if an attack guesses "trained on" whenever a classifier is right, the share of training points it flags minus the share of other points it flags equals the classifier's generalization gap[a] in accuracy[b]. The network of Solution 3.1 is right on 100% of its training digits and 92% of the others, so that guess is right 54% of the time on a half-and-half set. A network that overfits[c] gives away more about its training set[d].
Large language models can give away the training text itself, as Ethics and Fairness[e] in the DATA 471 notes describes; Carlini et al. [6] found text that appeared in only one document of GPT-2's training data[f] among its outputs.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-generalization-gap
[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-overfitting
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[e] https://morrison-lab.github.io/lds/chapters/ethics.html#misinformation-and-privacy
[f] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set

4 What training costs

4.1 Energy for GPT-3

Exercise 4.1 (Energy to train GPT-3)**.** Table 4 of Patterson et al. [7] gives these figures for training GPT-3:

- 3.14×10^{23} floating-point operations in all;
- 24.6×10^{12} operations a second on each of its V100 GPUs;
- 330 watts drawn by each GPU, counting its share of the memory, fans and host computer.

How much electrical energy did the GPUs use?

Solution

Solution 4.1.

```
flops, flops_per_second, watts = 3.14e23, 24.6e12, 330
gpu_seconds = flops / flops_per_second
gpu_years = gpu_seconds / (365.25 * 24 * 3600)
days_on_10k = gpu_seconds / 10_000 / (24 * 3600)
joules = gpu_seconds * watts
gpu_mwh = joules / 3.6e9


def sci(value):
    # A number as LaTeX scientific notation with 3 significant digits.
    mantissa, exponent = f"{value:.2e}".split("e")
    return f"${mantissa} \\times 10^{{{int(exponent)}}}$"


seconds_text, joules_text = sci(gpu_seconds), sci(joules)
print(seconds_text, gpu_years, days_on_10k, joules_text, gpu_mwh)
```

```
$1.28 \times 10^{10}$ 404.473966406711 14.773411623005119 $4.21 \times 10^{12}$ 1170.0542005420054
```

Dividing the operations by the rate gives 1.28×10^{10} GPU-seconds, about 404 GPU-years, or 14.8 days on 10,000 GPUs. At 330 watts each, the GPUs used 4.21×10^{12} joules. A megawatt-hour is 3.6×10^{9} joules, so that is 1,170 megawatt-hours.

Definition 4.1 (Power usage effectiveness (PUE))**.** The **power usage effectiveness**, or **PUE**, of a datacenter is the energy it uses in all divided by the energy its computing equipment uses [7, Sec. 2.3].

Example 4.1 (Two datacenters)**.** Patterson et al. [7, Sec. 2.3] give 1.58 as the average PUE of datacenters in the United States in 2018, and 1.11 for the Google datacenter in Iowa where they trained one of their models. For each kilowatt-hour its computers used, the Iowa datacenter used 1.11 in all; the other 0.11 was overhead beyond powering the computing equipment.

Exercise 4.2 (Carbon from training GPT-3)**.** Table 4 of Patterson et al. [7] also gives:

- a PUE of 1.10 for the datacenter that trained GPT-3;
- 0.429 kilograms of carbon dioxide equivalent per kilowatt-hour, the average for electricity in the United States that they use for GPT-3.

How many tonnes of carbon dioxide equivalent did training GPT-3 emit?

Solution

Solution 4.2.

```
pue, kg_co2e_per_kwh = 1.10, 0.429
total_mwh = gpu_mwh * pue
tonnes_co2e = total_mwh * 1000 * kg_co2e_per_kwh / 1000
round_trips = tonnes_co2e / 180
print(total_mwh, tonnes_co2e, round_trips)
```

```
1287.059620596206 552.1485772357723 3.067492095754291
```

Multiplying the 1,170 megawatt-hours of Solution 4.1 by the PUE gives 1,287 megawatt-hours for the whole datacenter, the estimate Patterson et al. [7] report. At 0.429 kilograms per kilowatt-hour, that is 552 tonnes of carbon dioxide equivalent, about 3 times the 180 tonnes they give for a jet's round trip between San Francisco and New York.

Remark (What sweeps cost)

Remark 4.1 (What sweeps cost). A sweep multiplies the cost of training by the number of trials. Strubell et al. [8] add up one research project that ran 123 small hyperparameter grid searches, 4,789 training runs in all, which used 27 GPU-years over six months. They recommend that papers report training time and how sensitive a model is to its hyperparameters [8, Sec. 5], so that readers can weigh what a result cost. Patterson et al. [7] find that the choice of model, datacenter and processor can change the carbon footprint of training by up to 100 to 1,000 times. Running Deep Learning Experiments shows how to plan a sweep so that each trial answers a question.

5 Documenting a model

5.1 What a user needs to know

Exercise 5.1 (What a user needs to know)**.** The network of Solution 2.1 trained on every eight is about to be shared. What should someone deciding whether to use it be told?

Solution

Solution 5.1. What it is, what it was trained on, what it is meant for, how well it does, on which inputs it does worse, and what it was never tested on. A single accuracy[a] answers only the fourth. Its training set[b] and validation set[c] hold digits from the same 13 writers, so how well it reads another person's handwriting is unknown, and its users should be told so.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-validation-set

Definition 5.1 (Model card)**.** A **model card** is a short document released with a trained model that reports its intended uses and its performance, including disaggregated evaluations over the groups that matter for those uses [2]. Mitchell et al. [2, Sec. 4] suggest nine sections:

- model details;
- intended use;
- factors, the groups or conditions the performance may depend on;
- metrics;
- evaluation data;
- training data;
- quantitative analyses;
- ethical considerations;

- caveats and recommendations.

Example 5.1 (A model card for the digit network)**.**

```
n_params = sum(p.numel() for p in full_net.parameters())
print(n_params, full_acc, full_by_digit.min(), full_by_digit.max(), most_images)
```

```
4810 0.9598494172096252 0.8947368 1.0 94
```

A model card for the network of Solution 2.1 trained on every eight:

- **Model details:** a network with one hidden layer of 64 ReLU units, 4,810 parameters[a], trained by full-batch Adam[b] for 300 steps, for these notes.
- **Intended use:** teaching. It is not meant for reading real handwriting, such as on forms or checks.
- **Factors:** the digit, and the writer, which the data do not record.
- **Metrics:** accuracy[c], overall and for each digit.
- **Evaluation data:** 797 random images from `data/optdigits-test.csv`[d] [1].
- **Training data:** the other 1,000 images of the same file, from the same 13 writers.
- **Quantitative analyses:** 96% correct overall, and from 89% on 3s to 100% on 7s (Table 1).
- **Ethical considerations:** the images record the handwriting of 13 people and nothing else about them.
- **Caveats and recommendations:** untested on other writers' handwriting; each digit has at most 94 validation images, so the accuracy for each digit could move by a few percentage points with a different split.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/mds/optimization-gradient-descent.html#def-adam
[c] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[d] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv

Definition 5.2 (Datasheet)**.** A **datasheet** for a data set is a document that answers questions about seven topics [9, Sec. 3]:

- why the data set was made;
- what it holds;
- how it was collected;
- how it was cleaned and labelled;
- what it should and should not be used for;
- how it is distributed;
- who maintains it.

Example 5.2 (A datasheet for the digits)**.** Part of a datasheet for the digits of this page, from the description that comes with them [1]:

- **Why:** the description does not say; the first use it lists is a 1995 thesis on combining classifiers to recognize handwritten digits.
- **What:** 5,620 images of digits in all, each an 8 by 8 grid of counts from 0 to 16, with its digit as the label. 43 people wrote them: 30 wrote the 3,823 training images, and 13 others wrote the 1,797 test images used on this page. No other facts about the writers are recorded.
- **Collection:** the digits were written on a preprinted form, and programs from the United States National Institute of Standards and Technology turned them into 32 by 32 bitmaps.
- **Cleaning:** each 32 by 32 image was cut into 4 by 4 blocks, and the number of on pixels in each block counted.
- **Uses:** the authors kept the test writers apart from the training writers, to measure accuracy[a] on new writers.
- **Distribution:** the UCI Machine Learning Repository, under a CC BY 4.0 license.

The description does not say who maintains the data. A model card for a network trained on these digits can point to this datasheet for its training and evaluation data.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

5.2 Further reading

Read

- Mitchell et al. [2] proposed model cards.
- Gebru et al. [9] proposed datasheets.
- Carlini et al. [6] extracted training data from GPT-2.
- Patterson et al. [7] estimated the energy and carbon cost of training large language models.

6 References

[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[7].

[2] M. Mitchell *et al.*, "Model cards for model reporting," in *Proceedings of the conference on fairness, accountability, and transparency*, 2019, pp. 220–229. doi: 10.1145/3287560.3287596[8].

[3] J. Buolamwini and T. Gebru, "Gender shades: Intersectional accuracy disparities in commercial gender classification," in *Proceedings of the 1st conference on fairness, accountability and transparency*, in Proceedings of machine learning research, vol. 81. 2018, pp. 77–91. Available: https://proceedings.mlr.press/v81/buolamwini18a.html

[4] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE symposium on security and privacy*, 2017, pp. 3–18. doi: 10.1109/SP.2017.41[9].

[5] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *2018 IEEE 31st computer security foundations symposium*, 2018, pp. 268–282. doi: 10.1109/CSF.2018.00027[10].

[6] N. Carlini *et al.*, "Extracting training data from large language models," in *30th USENIX security symposium*, 2021, pp. 2633–2650. Available: https://arxiv.org/abs/2012.07805

[7] D. Patterson *et al.*, "Carbon emissions and large neural network training." 2021. Available: https://arxiv.org/abs/2104.10350

[8] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019, pp. 3645–3650. doi: 10.18653/v1/P19-1355[11].

[9] T. Gebru *et al.*, "Datasheets for datasets." 2021. Available: https://arxiv.org/abs/1803.09010

[7] https://doi.org/10.24432/C50P49
[8] https://doi.org/10.1145/3287560.3287596
[9] https://doi.org/10.1109/SP.2017.41
[10] https://doi.org/10.1109/CSF.2018.00027
[11] https://doi.org/10.18653/v1/P19-1355